

AWS Cloud Architect — Technikai interjú felkészítő

Principal Cloud Engineer szint · Kérdés-válasz tudástár

Fókuszterületek: AWS alapok & architektúra · IaC · Konténerek & orchestration · CI/CD · Observability · Security · Networking · GenAI infra

Az alábbi kérdések azt a fajta **technikai mélységet firtató kérdést** reprezentálják, amivel az interjúztató a tudásod ellenőrzi — nem az önéletrajzodat. A válaszok a lényegre és a trade-offokra vannak sűrítve, arra, amit hangosan elmondanál. A szakkifejezések szándékosan angolul maradnak, mert az interjú is így fog menni.

— Narancs csík = erre a szintre kiemelten valószínű / core kérdés.

AWS alapok & architektúra

compute · storage · databases · Well-Architected

K Hogyan terveznél egy magas rendelkezésre állású, multi-AZ webalkalmazást AWS-en?

Legalább 2 AZ-ra szétosztva, egy **ALB** mögött; stateless app réteg **Auto Scaling Group**-ban; managed adatréteg **RDS Multi-AZ**-val (szinkron standby) vagy Aurora-val. Statikus tartalom **S3 + CloudFront**-ra. A session/state külső tárba kerül (Elasticache, DynamoDB), így bármelyik instance eldobható. A health check + ASG automatikusan lecseréli a leállt node-okat. Egy AZ kiesését így már túléli; egy Region kiesés cross-Region replikációt + Route 53 failovert igényel.

K Vedd végig a Well-Architected Framework 6 pillérét, és egy ellentmondást kettő között.

Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability. Klasszikus feszültség:

Reliability vs Cost — a Multi-AZ RDS megduplázza a standby költséget, a cross-Region DR pedig többszörözi. Ezt nem egy default-tal oldod fel, hanem az üzleti RTO/RPO alapján. Másik: **Performance vs Cost** — provisioned kapacitás vs on-demand/serverless.

K Mikor választanál ECS Fargate-et, EKS-t, illetve Lambdát egy workload-hoz?

Lambda: event-driven, tüskés, rövid (<15 perc) feladatok, nulla infra-üzemeltetéssel.

Fargate: konténeres, hosszan futó service-ek, serverless konténer node-menedzsment nélkül. **EKS:** ha kell a Kubernetes ökoszisztéma (operátorok, Helm, portabilitás, komplex scheduling) vagy multi-cloud stratégia — cserébe felvállalod az üzemeltetési terhet. A döntést a kontroll vs üzemeltetési teher határozza meg, plusz hogy a csapat beszéli-e már a Kuberneteset.

K Mi a különbség EBS, EFS és S3 között — és mikor melyik illik?

EBS: block storage, egy instance-hoz (vagy io2 Multi-Attach), mint egy virtuális lemez — adatbázisok, boot volume-ok.

EFS: NFS megosztott fájlrendszer, több instance egyszerre csatolja, elasztikusan skálázódik. **S3:** object storage HTTP felett, gyakorlatilag végtelen, 11 kilences durability — assetek, backupok, data lake-ek, lo-

gok. Nem fájlrendszer; nincs helyben szerkesztés.

K Hogyan választasz RDS, Aurora és DynamoDB között?

RDS: sima relációs, ismert engine-ek, közepes skála. **Aurora:** relációs, de nagyobb throughput, gyorsabb failover, 128 TB-ig automatikus storage-skálázás, 15-ig read replica kell. **DynamoDB:** key-value/document óriási skálán, egyszámjegű ms latency, serverless — de a hozzáférési mintákra kell tervezni, nem normalizált táblákra. A relációs joinok / ad-hoc lekérdezések visszavisznek RDS/Aurorára.

K S3 storage class-ok és lifecycle stratégia a költségre?

Standard → Standard-IA (ritka hozzáférés) → Glacier Instant/Flexible → Deep Archive, hozzáférési gyakoriság szerint. **Lifecycle szabályok**-kal automatikusan mozgatod az objektumokat kor alapján, és **Intelligent-Tiering**-et használsz, ha ismeretlen a hozzáférési minta. Figyelj a retrie-

val díjakra és a minimum tárolási időre a hideg tereken.

Networking & VPC

subnetek · routing · kapcsolat · DNS

K Tervezz egy biztonságos VPC felépítést egy 3-rétegű alkalmazáshoz.

Public subnetek: csak ALB + NAT Gateway. **Private subnetek:** app réteg, public IP nélkül, kifelé NAT-on át. **Isolated/private subnetek:** adatbázis réteg, egyáltalán nincs internet route. Rétegenként security group, amelyek egymást SG ID alapján hivatkozzák (nem CIDR-rel), így csak az ALB SG éri el az app SG-t, csak az app SG a DB SG-t. NACL-ok durva, stateless végső védvonalként. A subneteket AZ-k közt szétosztod.

K Security Group vs NACL — a lényegi különbségek?

Security Group: stateful (a visszatérő forgalom automatikusan engedélyezett), instance/ENI szinten, csak allow szabályok.

NACL: stateless (mindkét irányt engedni kell), subnet szinten, támogat explicit deny-t, szabálysorszám szerint kiértékelve. Az SG az elsődleges kontroll; a NACL a subnet-szintű tömbös tiltásokra jó.

K Hogyan ér el a forgalom egy private subnetben lévő instance-t, és az hogyan éri el az internetet?

Befelé: csak valamin keresztül, ami public subnetben van (ALB, bastion) — az IGW-től nincs közvetlen route. Kifelé: a private subnet route table a `0.0.0.0/0`-t egy public subnetben lévő **NAT Gateway**-re küldi, amelynek van tényleges IGW route-ja.

AWS API hívásokhoz inkább **VPC End-point** (Gateway az S3/DynamoDB-hez, Interface/PrivateLink a többihez), hogy teljesen kihagyd a NAT-ot — olcsóbb és a forgalom nem megy ki a publikus internetre.

K Milyen lehetőségek vannak on-prem és AWS összekötésére, és mi a trade-off?

Site-to-Site VPN: gyors, olcsó, publikus interneten át, titkosított, változó latency. **Di-**

Direct Connect: dedikált privát vonal, konzisztens alacsony latency és sávszélesség, de átfutási idő és költség. Gyakori minta: Direct Connect VPN backuppal a rezilienciáért. Sok VPC esetén egy **Transit Gateway** hubon terminálsz.

Infrastructure as Code

Terraform · CloudFormation · state · modulok

K Hogyan kezeled a Terraform state-et biztonságosan csapatban?

Remote backend — **S3 a state-nek + DynamoDB a state lockinghoz** (vagy az újabb S3-natív locking). Versioning + encryption a bucketen. A state sose megy gitbe (secreteket tartalmaz). State-et környezetenként/komponensenként szeparáld, hogy egy `dev` apply ne érhesse a `prod`-hoz. A `terraform plan` a CI-ban kötelező review kapu.

K Terraform vs CloudFormation vs Pulumi — hogyan választasz?

Terraform: multi-cloud, hatalmas provider ökoszisztéma, HCL, érett modulok — agency/multi-client munkára a default. **CloudFormation:** AWS-natív, nincs kezelendő state fájl (az AWS tartja), szoros service-integráció, beépített drift detection.

Pulumi: valódi nyelvek (TS/Python/Go) — jó, ha a csapat loopokat/absztrakciókat akar és az app-nyelvi tooling újrahasznosítását. Consultancynél, ahol vegyes klienseket szolgálsz ki, a Terraform portabilitása általában nyer.

K Mitől jó egy újrahasznosítható Terraform modul?

Egy tiszta felelősség, értelmes defaultok, jól típusozott, validált változók, értelmes outputok, semmi hardcode-olt account/region/env érték, és pinnelt provider verziók. A modult verziózd (registry vagy git tag), hogy a fogyasztók tudatosan frissítsenek. Ne over-parameterizáld — egy 80 változós modul csak Terraform plusz körökkel.

K Hogyan kezeled a secreteket IaC pipeline-okban?

Sose kódban vagy state-ben plaintextként. Runtime-ban húzod **Secrets Manager**-ból vagy **SSM Parameter Store**-ból (SecureString); CI változókon vagy OIDC-vel assumeolt role-okon át injektálsz. A Terraform a hivatkozott értékeket akkor is beleírja a state-be, ezért a state-et érzékenyként kezelsz (titkosítva, hozzáférés-korlátozva), és inkább maga az *alkalmazás* húzza a secretet közvetlenül, nem a Terraformon keresztül.

K Hogyan észleled és javítod a konfigurációs driftet?

A **terraform plan** (vagy CloudFormation drift detection) ütemezetten jelzi a pipeline-on kívüli változásokat. Javítás: újra-applylsz az IaC-t, vagy visszaimportálsz a változást a configba. A legjobb hosszú távú javítás a megelőzés: prodban korlátozod a console írási jogot, hogy minden változás a pipeline-on menjen át.

Konténerek & orchestration

Docker · Kubernetes · ECS/EKS · skálázás

K Magyarázd el a fő Kubernetes objektumokat, és hogyan ér el egy request egy podot.

A **Pod** (legkisebb deployolható egység) egy **Deployment** alatt fut (ReplicaSet-en át) a rollouthoz/skálázáshoz. Egy **Service** stabil virtuális IP-t/DNS-t ad és label selector alapján load-balancol a podok közt. A külső forgalom egy **Ingress**-en (+ controller, pl. ALB Controller) → Service → kube-proxy → pod úton lép be. A **ConfigMap/Secret** adja a configot; a **namespace** izolálja a tenanteket.

K Hogyan működik az autoscaling Kubernetesben EKS-en?

Két réteg: a **HPA** a pod replikákat skálázza CPU/memória/custom metrika alapján; a **Cluster Autoscaler** (vagy **Karpenter**) node-okat ad/vesz, ha egy pod nem ütemezhető. EKS-en ma a Karpenter a preferált — gyorsan provisionál megfelelő mére-

tű node-okat és bin-packel a költségért. A VPA igazíthatja a pod requesteket, de ugyanazon a metrikán ütközik a HPA-val.

K Hogyan teszel egy Docker image-et production-readyvé?

Multi-stage build, hogy a runtime image picit maradjon; minimális/distroless base; **non-root** futtatás; pinnelt verziók; `.dockerignore` a context szűkítésére és a secret-szivárgás ellen; egy process konténerenként; health check deklarálva. Az image-et scanned (Trivy/ECR scanning) a CI-ban push előtt.

K ECS vs EKS üzemeltetési szempontból?

ECS: AWS-proprietary, sokkal egyszerűbb, mély AWS integráció, kevesebb üzemeltetés — remek, ha nem kell Kubernetes.

EKS: standard Kubernetes API, portábilis, gazdag ökoszisztéma, de tiéd az upgrade, az add-onok és a több mozgó alkatrész. ECS-t sebességért és kisebb üzemeltetésért; EKS-t portabilitásért és ökoszisztéma-igényért.

K Hogyan előzik meg a rolling deploymentek és a health checkek a downtime-ot?

Az új podok/taskok elindulnak, és át kell menniük a **readiness** checken, mielőtt forgalmat kapnának; a régiek csak akkor drainelnek, ha az újak egészségesek (maxUnavailable/maxSurge szabja a tempót). A bukott readiness megállítja a rolloutot ahelyett, hogy hibát szolgálna ki. Párosítsd **liveness** checkkel a beragadt konténerek újraindítására. Lehetővé teszi a blue/green vagy canary megközelítést traffic shifting-gel.

CI/CD & deployment

GitLab CI · GitHub Actions · pipeline-ok · stratégiák

K Tervezz egy CI/CD pipeline-t commitól productionig.

Commit → **lint + unit tesztek** → build artifact/image → **scan** (SAST, image, IaC) → push registrybe → deploy stagingre → integrációs/smoke tesztek → **manuális approval kapu** → deploy prodra (rolling/

canary) → post-deploy ellenőrzés + auto-rollback hibára. Tartsd gyorsnak a visszajelzésért; bukj korán az olcsó checkeken.

K Blue/green vs canary vs rolling — trade-offok?

Rolling: fokozatos csere, olcsó, de rövid ideig vegyes verzió szolgál ki forgalmat.

Blue/green: teljes párhuzamos környezet, azonnali átkapcsolás + azonnali rollback, de ~2× infra a váltás alatt. **Canary:** a forgalom kis %-át küldöd az új verzióra, figyeled a metrikákat, majd rámpázol — kockázatra a legbiztonságosabb, jó observability és traffic-splitting kell hozzá. A blast-radius tűréshatár és a költség alapján választasz.

K Hogyan autentikálsz egy CI pipeline-t AWS felé long-lived kulcsok nélkül?

OIDC federation: a GitLab/GitHub identity providerként lép fel; a pipeline STS-en át rövid életű credentialekkel assumeol egy IAM role-t, amit a trust policy szűkít (pl. konkrét repo/branch). Nincs statikus access key rotálni vagy szivárogtatni. Ez a

modern best-practice válasz, és erős jelzés interjún.

K Hogyan tartod hatékonyan egy monorepo pipeline-ját?

Change detection, hogy csak az érintett service-ek buildeljenek/deployoljanak; caching (függőségek, layerek, Terraform providerek); párhuzamos jobok; újrahasznosítható pipeline template-ek/komponensek. Bukj korán az olcsó checkek előre rendezésével.

Observability & üzemeltetés

metrikák · logok · trace-ek · alerting · incidensek

K Mi az observability három pillére, és hogyan illeszkednek össze?

Metrics (numerikus idősor — mi a baj, olcsó tárolni), **logs** (diszkrét események — részletek/kontextus), **traces** (egy request útja a service-eken át — hol a latency/hiba). Folyamat: egy metrika-alert megszólal → a trace-ek lokalizálják a bukott hopot → a logok elmagyarázzák a root cause-t.

AWS-en: CloudWatch metrics/logs, X-Ray vagy OpenTelemetry a tracinghez; Prometheus/Grafana/ELK alternatív stackként.

K Hogyan állítasz be alerteket, amik actionable-ek, nem zajosak?

Symptomokra/SLO-kra alertelj (error rate, latency, saturation), ne nyers okokra; csak user-impacting feltételre pagelj. Használd a **négy golden signal**-t (latency, traffic, errors, saturation). Küszöbök időablakkal a flapping ellen, súlyossági szintek (page vs ticket), és runbook link minden alerten. Öld ki azt az alertet, amire senki nem reagál.

K Definiáld az SLI-t, SLO-t, SLA-t, és hogyan vezérel az error budget.

SLI: mért indikátor (pl. sikeres requestek %-a). **SLO**: belső cél erre az SLI-re (pl. 99,9%). **SLA**: külső, szerződéses ígélet (általában lazább, büntetéssel). A 100% alatti rés az **error budget**: ha gyorsan égeted, befagyasztod a kockázatos release-eket és a reliabilityt priorizálod; ha egészséges, gyorsabban szállítasz.

K Vedd végig az incident response folyamatodat.

Detect (alert/monitor) → súlyosság deklarálása → **incident commander** kijelölése → először mitigálsz (rollback, skálázás, failover), csak utána root-cause → státusz kommunikáció a stakeholderek felé → resolve → **blameless post-mortem** timeline-nal és action itemekkel. A szisztematikus okot és a detektálási rést javítod, nem csak a tünetet.

Security

IAM · encryption · secrets · least privilege

K Magyarázd el, hogyan értékelődik ki egy IAM policy (role-ok, policyk, precedencia).

Alap: **deny**. Egy request csak akkor engedélyezett, ha egy vonatkozó policy explicit allow-t ad és egyik policy sem ad explicit deny-t — az **explicit deny mindig nyer**. A tényleges jogosultság az identity policyk, resource policyk, permission boundaryk és SCP-k (Organizations) metszete. Inkább

role-ok (temporary STS credentialek), mint IAM userek statikus kulccsal.

K Hogyan valósítod meg a **least privilege-et** a gyakorlatban?

Nulláról indulsz, és csak a szükséges action/resource párokat adod hozzá; resource ARN-re és feltételekre szűkíted (pl. tag, source VPC, MFA). **Permission boundary**-val korlátozod, mit adhatnak a csapatok, **Access Analyzer**-rel keresed a túl tág vagy külső hozzáférést, és a CloudTrail/last-accessed adat alapján nyesed a nem használt jogokat. Prodban kerüld a -ot a policykban.

K Encryption at rest vs in transit AWS-en, és a KMS szerepe.

In transit: TLS mindenhol (ALB/CloudFront certek ACM-en át). **At rest:** SSE bekapcsolva S3/EBS/RDS-en, **KMS** kulcsokkal a háttérben. A KMS kezeli a kulcs-életciklust, rotációt és grantokat; **customer-managed key**-t használsz, ha kontroll kell a policy/rotáció/audit felett, és key policyk + grantok szabják meg, ki dekódolhat. Az envelope

encryption a data key-eket a CMK-val wrappelve tartja.

K Secrets Manager vs SSM Parameter Store?

Secrets Manager: secretekre tervezve — automatikus rotáció, cross-Region replikáció, magasabb költség. **Parameter Store:** config + secret (SecureString), olcsóbb/free tier, nincs natív rotáció. Secrets Managert akkor, ha rotáció kell (DB credek); Parameter Store-t általános confighoz és egyszerűbb secretekhez.

K Hogyan biztonságosítasz egy multi-account AWS setupot?

AWS Organizations SCP-kkel guardrail-ként (region tiltás, CloudTrail kikapcsolásának tiltása stb.), külön account környezetként/workloadonként a blast-radius izolációért, központosított logging + security tooling dedikált accountokban, és **Control Tower / Landing Zone** a konzisztens account-provisioninghez. Cross-account hozzáférés assumeolt role-okkal, sose megosztott credentiallel.

AI / GenAI infrastruktúra

Bedrock · RAG · vektor DB-k · inference

K Magyarázd el egy RAG rendszer architektúráját AWS-en.

Ingest: dokumentumok → chunkolás → **embedding** (Bedrock/Titan vagy hasonló) → vektorok tárolása egy **vektor DB**-ben (OpenSearch k-NN, Aurora pgvector vagy managed store). Query: a user kérdését embeddeled → similarity search a top-k chunkra → beinjektálsz contextként a promptba → az LLM (**Bedrock**-on át) grounded választ generál. Frissességet és forrás-groundingot ad újratanítás nélkül, és forrás-hivatkozást is enged.

K Miért kell vektor adatbázis, és mik a trade-offok a választásban?

Gyors **approximate nearest-neighbor** keresést csinál magas dimenziós embeddingeken — normál indexekkel megoldhatatlan. Trade-offok: recall vs latency (ANN algoritmus/paraméterek, pl. HNSW), managed vs self-hosted üzemeltetés, költség ská-

lán, és hogy újrahasznosítod-e a meglévő infrát (pgvector Aurorán) vs dedikált engine (OpenSearch). Skála, meglévő stack és üzemeltetési étvágy alapján választasz.

K Hogyan teszed az LLM inference-t skálázhatóvá és költség-kontrollálttá?

Inkább managed **Bedrock**, hogy elkerüld a GPU-üzemeltetést; on-demand a tükés terhelésre, provisioned throughput a stabil, nagy volumenre. Költséget **cachinggel** (semantic/response cache), prompt/context trimmeléssel, a modell feladathoz méretezésével (kis modell könnyű hívásra), token limitekkel és async batcheléssel kontrollálsz. A per-request token-költséget elsődleges metrikaként figyeled.

K Mik a fő kockázatok, ha egy GenAI rendszert productionbe teszel?

Prompt injection és data exfiltration, hallucináció (RAG groundinggal + guardrailekkel mérsékled), érzékeny adat szivárgása a promptba/logokba, költség-elszállás korlátlan használatból, és latency. Adj hozzá input/output guardraileket, PII redaction-t,

rate limitet, és a válaszmínőség evaluation/monitoringját — kezeld úgy, mint egy untrusted-input rendszert.

Szenárió- & trade-off kérdések

nyílt végű · „hogyan...”

K Egy kliens oldala lelassul forgalmi túske alatt. Hogyan diagnosztizálsd és javítod?

Diagnózis a **golden signal**-okkal: nézd, hol jelenik meg a latency/saturation (LB, app, DB). Gyakori javítások: horizontális **auto-scaling** az app rétegen, **CloudFront + caching** a statikus/edge tartalomra, **read replica** vagy connection pooling a DB nyomásra, és egy async queue (SQS) a túskek elnyelésére. Load-tesztel validálsz. Különítsd el a skálázási problémát az inefficienciától (N+1 query), mielőtt hardvert dobnál rá.

K Hogyan terveznél egy legacy on-prem app AWS-re migrálását?

Felmérés, és komponensenként **7 R** stratégia (rehost, replatform, refactor...). Alacsony kockázatú pilottal indulsz, felállítod a landing zone-t/networkinget (VPC, Direct Connect/VPN), migrálsz az adatot (DMS) cutover tervvel és rollbackkel, párhuzamosan futtatod a validálásért, majd DNS-t váltasz. Üzleti érték és kockázat szerint priorizálsz; nem refaktorálsz mindent az első napon.

K Hogyan állsz a disaster recoveryhez, és hogyan választasz RTO/RPO stratégiát?

Négy minta költség/sebesség szerint:

Backup & Restore (legolcsóbb, lassú), **Pilot Light**, **Warm Standby**, **Multi-Site Active/Active** (leggyorsabb, legdrágább). Az üzleti **RTO** (milyen gyorsan állsz helyre) és **RPO** (elfogadható adatvesztés) választja a mintát — a túltervezett DR pénzt éget, az alultervezett kockáztatja az üzletet. Rendszeresen teszteld a failvert; egy nem tesztelt DR terv nem is létezik.

K Egy kliens 30%-kal vágná az AWS számláját. Hol nézel körül?

Right-sizeolod a túl-provisionált instance-eket/volume-okat; **Savings Plans / Reserved Instances** a stabil baseline terhelésre; **Spot** a hibatűrő/batch munkára; S3 lifecycle a hidegebb tierekre; törölöd az árva resource-okat (unattached EBS, idle NAT/ELB, régi snapshotok); NAT Gateway-eket konszolidálsz és VPC endpointot használasz; bekapcsolod a Cost Explorer/Compute Optimizer ajánlásokat. Tagelsz a cost allocationhöz, hogy lásd, hova megy tényleg a pénz.